

# INTERNSHIP PROJECT REPORT TASK-5

## Title

Create account on the below domain (link: <https://www.hackthebox.com/>) solve the Brutus lab. Based on brute force SSH note every step and tool you used to solve this lab and create a detailed report on it.

## 1. Introduction

As part of my internship at Elevance Skills Technology, I worked on a cybersecurity investigation project using the Hack The Box Sherlock challenge named "Brutus". The purpose of this task was to analyze system logs and understand how an attacker gained access to a Linux server through an SSH brute-force attack.

During this project, I examined authentication records, tracked login activities and identified suspicious actions. This hands-on investigation helped me apply the cybersecurity concepts learned during the internship and improve my practical skills in Linux log analysis, incident response and digital forensics.

## 2. Background

Cybersecurity incidents often occur when attackers try to gain unauthorized access to computer systems. One common method is a brute-force attack, where multiple username and password combinations are repeatedly tested until a valid login is found. If successful, attackers can access sensitive information, modify system settings or perform other malicious activities.

The Brutus Sherlock challenge is based on a realistic scenario involving an SSH brute-force attack against a Linux server. The challenge provides log files containing evidence of the attack. By analyzing these logs, it is possible to identify how the attacker gained access, track their activities and understand the impact of the incident.

### 3. Learning Objectives

The purpose of this project was to gain practical knowledge of cybersecurity investigation and log analysis. Through the Brutus Sherlock challenge, I was able to understand how security incidents can be detected and analyzed using Linux system logs.

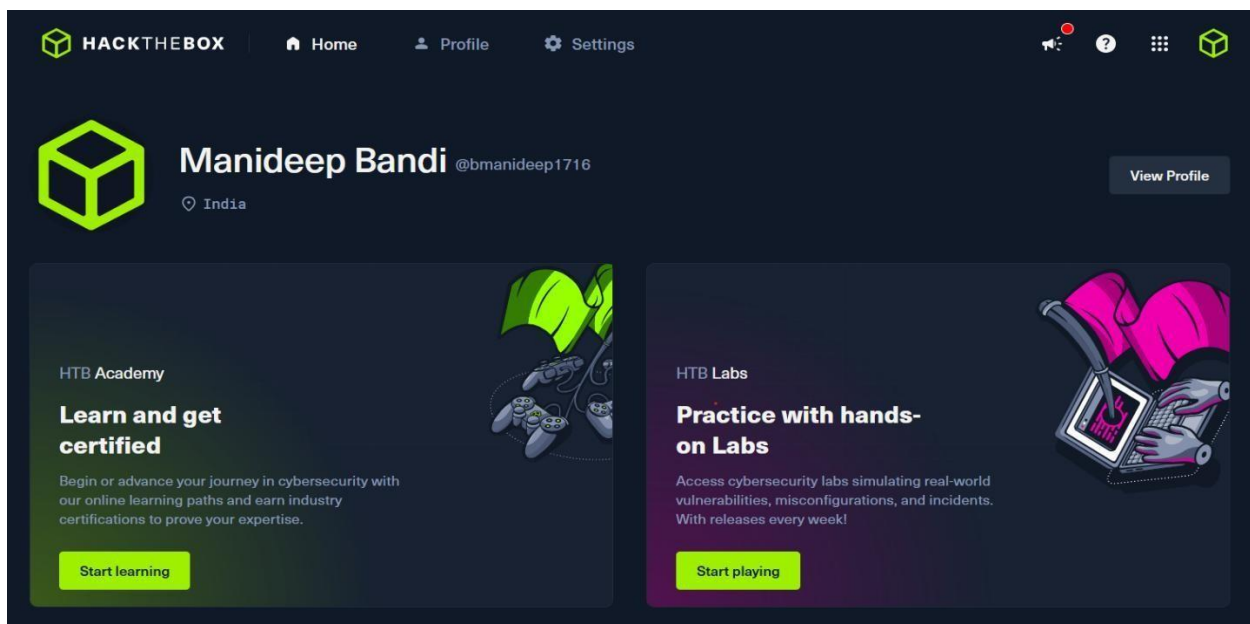
The main objectives of this project were:

- To understand how SSH brute-force attacks occur on Linux systems.
- To learn how to analyze authentication logs and identify suspicious activities.
- To investigate attacker actions using system log files.
- To improve Linux command-line and log analysis skills.
- To gain practical experience in digital forensics and incident response.

### 4. Activities and Tasks Performed

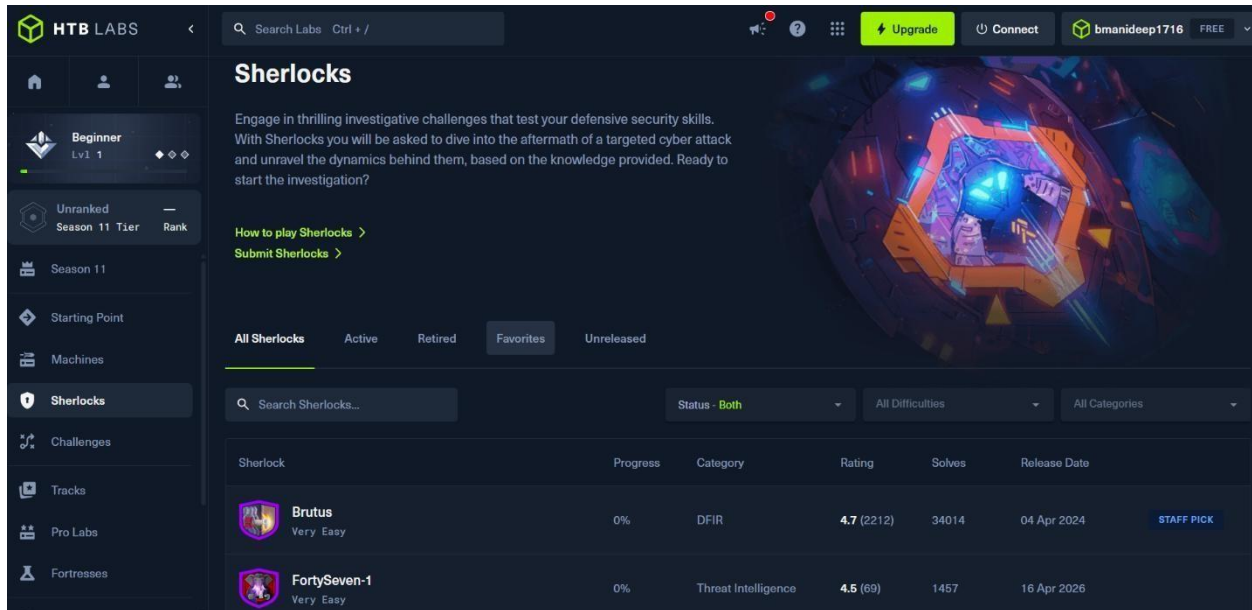
#### Step 1 - Hack The Box Account Setup

The first step of this project was creating and accessing a Hack The Box (HTB) account. After successfully registering on the platform, I logged in and verified my profile details. The HTB dashboard provided access to various cybersecurity learning resources, labs, and Sherlock challenges.



## Step 2 - Accessing the Brutus Sherlock Challenge

After accessing the Hack The Box platform, I navigated to the Sherlocks section, which contains various cybersecurity investigation challenges. I searched through the available challenges and selected the "Brutus" Sherlock challenge. Before starting the investigation, I reviewed the challenge information, difficulty level, and category to understand the task requirements and objectives.

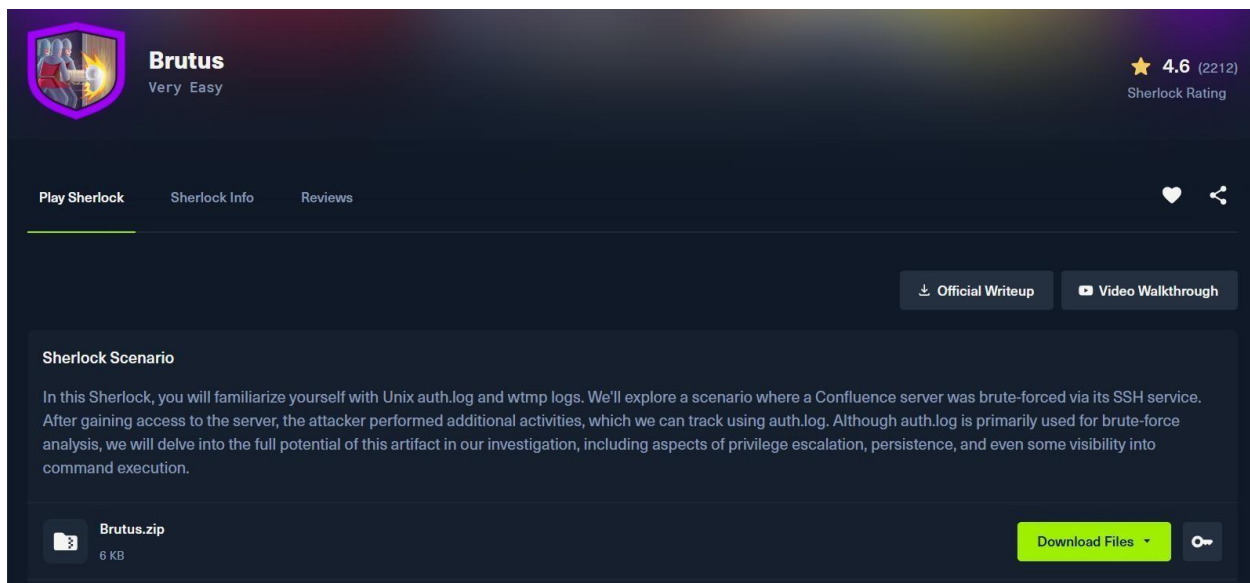


The screenshot shows the HTB Labs interface. The left sidebar contains navigation links: Home, Profile, and a list of challenges including Beginner (Lv1 1), Unranked (Season 11 Tier), Rank, Season 11, Starting Point, Machines, Sherlocks (selected), Challenges, Tracks, Pro Labs, and Fortresses. The main content area is titled "Sherlocks" and features a description: "Engage in thrilling investigative challenges that test your defensive security skills. With Sherlocks you will be asked to dive into the aftermath of a targeted cyber attack and unravel the dynamics behind them, based on the knowledge provided. Ready to start the investigation?". Below this are links for "How to play Sherlocks" and "Submit Sherlocks". A filter bar shows "All Sherlocks" selected, with tabs for Active, Retired, Favorites, and Unreleased. A search bar and filters for Status (Both), All Difficulties, and All Categories are present. A table lists the challenges:

| Sherlock                                                                                                             | Progress | Category            | Rating     | Solves | Release Date |            |
|----------------------------------------------------------------------------------------------------------------------|----------|---------------------|------------|--------|--------------|------------|
|  <b>Brutus</b><br>Very Easy        | 0%       | DFIR                | 4.7 (2212) | 34014  | 04 Apr 2024  | STAFF PICK |
|  <b>FortySeven-1</b><br>Very Easy | 0%       | Threat Intelligence | 4.5 (69)   | 1457   | 16 Apr 2026  |            |

## Step 3 - Starting the Brutus Challenge and Downloading Evidence Files

I opened the **Brutus** Sherlock challenge and carefully read the provided scenario to understand the incident being investigated. The scenario explained that a Linux server had been targeted through an SSH brute-force attack and that log analysis would be required to identify the attacker's actions. After understanding the objectives of the challenge, I downloaded the evidence files provided with the challenge. These files contained the system logs and artifacts needed for the investigation and answer extraction process.



The screenshot shows the interface for the 'Brutus' Sherlock challenge. At the top left is a shield icon with a red and yellow design. Next to it, the title 'Brutus' is displayed in a bold font, with 'Very Easy' underneath. On the top right, there is a star rating of 4.6 based on 2212 reviews, labeled 'Sherlock Rating'. Below the title, there are three tabs: 'Play Sherlock' (which is active), 'Sherlock Info', and 'Reviews'. To the right of these tabs are icons for a heart and a share symbol. Below the tabs, there are two buttons: 'Official Writeup' and 'Video Walkthrough'. The main content area is titled 'Sherlock Scenario' and contains a paragraph of text describing the challenge: 'In this Sherlock, you will familiarize yourself with Unix auth.log and wtmp logs. We'll explore a scenario where a Confluence server was brute-forced via its SSH service. After gaining access to the server, the attacker performed additional activities, which we can track using auth.log. Although auth.log is primarily used for brute-force analysis, we will delve into the full potential of this artifact in our investigation, including aspects of privilege escalation, persistence, and even some visibility into command execution.' At the bottom left, there is a file icon and the text 'Brutus.zip' with '6 KB' below it. To the right of this is a green 'Download Files' button and a key icon.

## Step 4 - Preparing the Investigation Environment

I started my Kali Linux virtual machine and prepared the environment for the investigation. Kali Linux was used as the working platform to analyze the evidence files, review logs, and perform the required forensic investigation. This setup ensured that all tools and files were ready before beginning the analysis.



## Step 5 - Extracting and Reviewing the Evidence Files

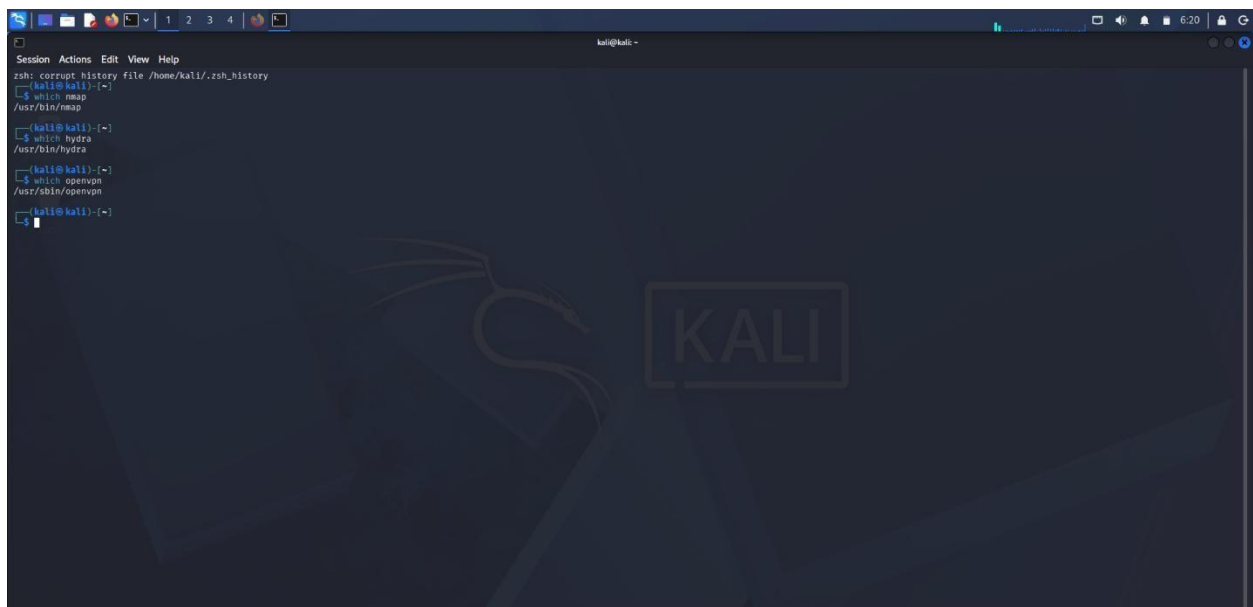
I extracted the downloaded **Brutus.zip** file and reviewed the files provided in the challenge. I identified the important log files and supporting scripts that would be used during the investigation. This helped me understand the available evidence before starting the analysis.

## Step 6 - Verifying Required Tools

I opened the terminal and checked whether the required tools were available in Kali Linux. I verified that the required Linux tools and utilities were available in Kali Linux before starting the investigation to ensure the environment was ready for cybersecurity and forensic analysis tasks.

### Tools Used-

- **Kali Linux** - Used as the operating system for performing the investigation and analysis.
- **7z** - The 7z tool was used to extract the contents of the downloaded ZIP file and access the evidence files needed for analysis.
- **grep** - The grep command was used to search specific keywords and events within the log files, helping to quickly identify important activities.
- **head** - The head command was used to view the first few lines of the log file and understand the initial events recorded in the logs.
- **tail** - The tail command was used to view the last few lines of the log file and examine the most recent activities.
- **utmp.py** - The Python script utmp.py was used to analyze the wtmp file and extract user login, logout, and session activity information from the system records.



```
zsh: corrupt history file /home/kali/.zsh_history
kali@kali:~$ which nmap
/usr/bin/nmap
kali@kali:~$ which hydra
/usr/bin/hydra
kali@kali:~$ which openvpn
/usr/sbin/openvpn
kali@kali:~$
```

## Step 7 - Preparing the Investigation Files

After preparing the investigation workspace, I created a dedicated folder named **Brutus-Lab** to store all files related to the Brutus challenge.

First, I created the folder using the command: **mkdir ~/Brutus-Lab**

Then, I navigated to the newly created folder: **cd ~/Brutus-Lab**

To verify the contents of the directory, I used: **ls -la**

After confirming that the folder was ready, I moved to the **Downloads** directory where the challenge files had been downloaded: **cd ~/Downloads**

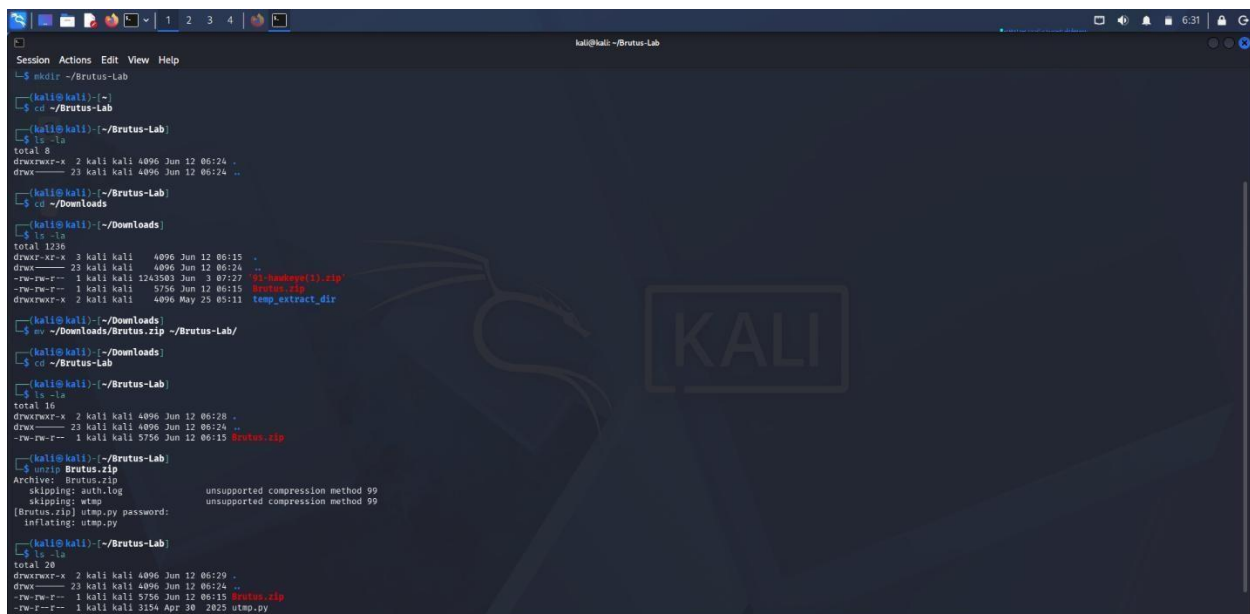
I listed the available files using: **ls -la**

Next, I moved the Brutus ZIP file into the **Brutus-Lab** folder using:  
**mv ~/Downloads/Brutus.zip ~/Brutus-Lab/**

After moving the file, I returned to the investigation directory: **cd ~/Brutus-Lab**

I then extracted the contents of the ZIP archive using: **unzip Brutus.zip**

Finally, I checked the folder contents again: **ls -la**



```
kali@kali: ~/Brutus-Lab
$ mkdir ~/Brutus-Lab
$ cd ~/Brutus-Lab
$ ls -la
total 8
drwxrwxr-x 2 kali kali 4096 Jun 12 06:24 .
drwx----- 23 kali kali 4096 Jun 12 06:24 ..
$ cd ~/Downloads
$ ls -la
total 1236
drwxr-xr-x 3 kali kali 4096 Jun 12 06:15 .
drwx----- 23 kali kali 4096 Jun 12 06:24 ..
-rw-rw-r-- 1 kali kali 1243503 Jun 3 07:27 'SI-hackney(1).zip'
-rw-rw-r-- 1 kali kali 5756 Jun 12 06:15 Brutus.zip
drwxrwxr-x 2 kali kali 4096 May 25 05:11 temp_extract_dir
$ mv ~/Downloads/Brutus.zip ~/Brutus-Lab/
$ cd ~/Brutus-Lab
$ ls -la
total 16
drwxrwxr-x 2 kali kali 4096 Jun 12 06:28 .
drwx----- 23 kali kali 4096 Jun 12 06:24 ..
-rw-rw-r-- 1 kali kali 5756 Jun 12 06:15 Brutus.zip
$ unzip Brutus.zip
Archive: Brutus.zip
  skipping: auth.log           unsupported compression method 99
  skipping: wtmp
[Brutus.zip] utmp.py password:
  inflating: utmp.py
$ ls -la
total 20
drwxrwxr-x 2 kali kali 4096 Jun 12 06:29 .
drwx----- 23 kali kali 4096 Jun 12 06:24 ..
-rw-rw-r-- 1 kali kali 5756 Jun 12 06:15 Brutus.zip
-rw-r--r-- 1 kali kali 3154 Apr 30 2025 utmp.py
```

## Step 8 - Extracting the Protected Evidence Files

While extracting the **Brutus.zip** file, I noticed that some files could not be extracted properly using the standard extraction method because of the compression format used in the archive. To resolve this issue, I updated the package information on Kali Linux using the command: **sudo apt update**

This command refreshed the package lists and ensured that the system had the latest information about available software packages.

Next, I verified the availability of the 7-Zip extraction utility using the command:  
**sudo apt install p7zip-full -y**

After confirming that 7-Zip was available, I extracted the archive using:  
**7z x Brutus.zip**

This command started the extraction process and prompted for the challenge password. After providing the password, the protected files inside the archive became accessible for further analysis.

```
kali@kali: ~/Brutus-Lab
$ sudo apt update
[sudo] password for kali:
Get:1 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [21.2 MB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:3 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [104 kB]
Get:4 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [189 kB]
Get:5 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [175 kB]
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free-firmware amd64 Packages [19.8 kB]
Get:8 http://mirrors.esto.network/kali kali-rolling/non-free-firmware amd64 Contents (deb) [38.9 kB]
Fetched 75.9 MB in 47s (1,629 KB/s)
325 packages can be upgraded. Run 'apt list --upgradable' to see them.

kali@kali:~/Brutus-Lab$ sudo apt install p7zip-full -y
Note, selecting '7zip' instead of 'p7zip-full'
7zip is already the newest version (26.01-dfsg-2).
7zip set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2 libavc1394-0 libbio-pty-perl libbmedcrypto16 libpostal1 libbsd12-classic libbsdium23 libteamdctl0 libxsltsec1-1 libxstring3
  libxsltsec1-openssl postgresql-common-dev python3-pluginbase python3-tz
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 325

kali@kali:~/Brutus-Lab$ 7z x Brutus.zip
7-Zip 26.01 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-04-27
64-bit locale=en_US.UTF-8 Threads:2 OPEN_MAX:4096, ASM

Scanning the drive for archives:
 1 file, 5756 bytes (6 KiB)

Extracting archive: Brutus.zip
--
Path = Brutus.zip
Type = zip
Physical Size = 5756

Enter password (will not be echoed):

Would you like to replace the existing file:
Path: ./utmp.py
Size: 3134 bytes (4 KiB)
Modified: 2025-04-28 04:51:29
```



## Step 9 - Examining the Authentication Log File

After extracting the contents of the **Brutus.zip** archive, I reviewed the extraction process to ensure that all files had been recovered successfully. After the extraction was completed, I checked the contents of the **Brutus-Lab** directory using the command: **ls -la**

This command displayed all files present in the investigation folder. The output confirmed that the important evidence files, including **auth.log**, **utmp** and **utmp.py** were available for analysis.

After confirming that the evidence files were available, I started reviewing the contents of the **auth.log** file to understand the login and authentication activities recorded on the server.

First, I checked the beginning of the log file using the command: **head auth.log**. This command displayed the first few entries in the log file. It helped me understand the log format and identify the types of events being recorded.

Next, I examined the most recent entries using the command: **tail auth.log**. This command displayed the last few lines of the log file, allowing me to review the latest events and observe activities related to user logins, session creation and privilege usage.

```
Session Actions Edit View Help
Modified: 2025-04-20 04:51:29
with the file from archive!
Path: utmp.py
Size: 3154 bytes (4 KiB)
Modified: 2025-04-20 04:51:29
? (Y)es / (N)o / (A)llways / (S)kip all / (U)to rename all / (Q)uit? A

Everything is OK

Files: 3
Size: 58201
Compressed: 5756

[kali@kali:~/Brutus-Lab]
$ ls -la
total 76
drwxrwxr-x 2 kali kali 4096 Jun 12 06:39 .
drwx----- 23 kali kali 4096 Jun 12 06:24 ..
-rw-rw-r-- 1 kali kali 43911 Mar  6 2024 auth.log
-rw-rw-r-- 1 kali kali 5756 Jun 12 06:15 Brutus.zip
-rw-rw-r-- 1 kali kali 3154 Apr 30 2025 utmp.py
-rw-rw-r-- 1 kali kali 11130 Mar  6 2024 utmp

[kali@kali:~/Brutus-Lab]
$ head auth.log
Mar  6 06:18:01 ip-172-31-35-28 CRON[1119]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:18:01 ip-172-31-35-28 CRON[1118]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:18:01 ip-172-31-35-28 CRON[1117]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:18:01 ip-172-31-35-28 CRON[1118]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:18:01 ip-172-31-35-28 CRON[1117]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:18:01 ip-172-31-35-28 CRON[1117]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:19:01 ip-172-31-35-28 CRON[1366]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:19:01 ip-172-31-35-28 CRON[1367]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:19:01 ip-172-31-35-28 CRON[1366]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:19:01 ip-172-31-35-28 CRON[1367]: pam_unix(cron:session): session closed for user confluence

[kali@kali:~/Brutus-Lab]
$ tail auth.log
Mar  6 06:39:38 ip-172-31-35-28 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by cyberjunkie(uid=1002)
Mar  6 06:39:39 ip-172-31-35-28 sudo: pam_unix(sudo:session): session closed for user root
Mar  6 06:40:01 ip-172-31-35-28 CRON[2783]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:40:01 ip-172-31-35-28 CRON[2784]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:40:01 ip-172-31-35-28 CRON[2783]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:40:01 ip-172-31-35-28 CRON[2784]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:41:01 ip-172-31-35-28 CRON[2790]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:41:01 ip-172-31-35-28 CRON[2797]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar  6 06:41:01 ip-172-31-35-28 CRON[2797]: pam_unix(cron:session): session closed for user confluence
Mar  6 06:41:01 ip-172-31-35-28 CRON[2796]: pam_unix(cron:session): session closed for user confluence

[kali@kali:~/Brutus-Lab]
```

## Step 10 - Investigating Failed Login Attempts

After reviewing the authentication log file, I analyzed the failed SSH login attempts to determine whether a brute-force attack had occurred and to identify the attacker's activity.

First, I counted the total number of failed password attempts using the command:  
**grep "Failed password" auth.log | wc -l**

This command searched for all failed password entries in the log file and displayed the total count. The output showed **48** failed login attempts.

Next, I viewed the first few failed login records using:

**grep "Failed password" auth.log | head**

This command displayed the earliest failed login attempts. The output showed repeated login attempts against different usernames such as **admin** and **backup** from the same source IP address.

Then, I examined the most recent failed login entries using:

**grep "Failed password" auth.log | tail**

This command displayed the last few failed login attempts. The output showed multiple failed authentication attempts targeting several user accounts, including **root**, from the same remote IP address.

By reviewing these records, I confirmed that the server was targeted by a brute-force attack in which the attacker repeatedly attempted different usernames and passwords until valid credentials were discovered.

```
Session Actions Edit View Help
Mar 06:18:01 ip-172-31-35-28 CRON[1117]: pam_unix(cron:session): session closed for user confluence
Mar 06:19:01 ip-172-31-35-28 CRON[1366]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:19:01 ip-172-31-35-28 CRON[1367]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:19:01 ip-172-31-35-28 CRON[1366]: pam_unix(cron:session): session closed for user confluence
Mar 06:19:01 ip-172-31-35-28 CRON[1367]: pam_unix(cron:session): session closed for user confluence

[kali@kali:~/Brutus-Lab]
$ tail auth.log
Mar 06:39:28 ip-172-31-35-28 sudo: pam_unix(sudo:session): session opened for user root(uid=0) by cyberjunkie(uid=1002)
Mar 06:39:39 ip-172-31-35-28 sudo: pam_unix(sudo:session): session closed for user root
Mar 06:40:01 ip-172-31-35-28 CRON[2783]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:40:01 ip-172-31-35-28 CRON[2784]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:40:01 ip-172-31-35-28 CRON[2783]: pam_unix(cron:session): session closed for user confluence
Mar 06:40:01 ip-172-31-35-28 CRON[2784]: pam_unix(cron:session): session closed for user confluence
Mar 06:41:01 ip-172-31-35-28 CRON[2796]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:41:01 ip-172-31-35-28 CRON[2797]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:41:01 ip-172-31-35-28 CRON[2797]: pam_unix(cron:session): session closed for user confluence
Mar 06:41:01 ip-172-31-35-28 CRON[2796]: pam_unix(cron:session): session closed for user confluence

[kali@kali:~/Brutus-Lab]
$ grep "Failed password" auth.log | wc -l
48

[kali@kali:~/Brutus-Lab]
$ grep "Failed password" auth.log | head
Mar 06:31:33 ip-172-31-35-28 sshd[2322]: Failed password for invalid user admin from 65.2.161.68 port 46392 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2331]: Failed password for invalid user admin from 65.2.161.68 port 46436 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2332]: Failed password for invalid user admin from 65.2.161.68 port 46444 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2332]: Failed password for invalid user admin from 65.2.161.68 port 46460 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2337]: Failed password for invalid user admin from 65.2.161.68 port 46468 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2341]: Failed password for invalid user admin from 65.2.161.68 port 46456 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2336]: Failed password for backup from 65.2.161.68 port 46512 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2336]: Failed password for backup from 65.2.161.68 port 46468 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2336]: Failed password for invalid user admin from 65.2.161.68 port 46422 ssh2
Mar 06:31:33 ip-172-31-35-28 sshd[2328]: Failed password for invalid user admin from 65.2.161.68 port 46390 ssh2

[kali@kali:~/Brutus-Lab]
$ grep "Failed password" auth.log | tail
Mar 06:31:39 ip-172-31-35-28 sshd[2390]: Failed password for invalid user svc_account from 65.2.161.68 port 46800 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd[2400]: Failed password for invalid user svc_account from 65.2.161.68 port 46854 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd[2399]: Failed password for root from 65.2.161.68 port 46852 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd[2407]: Failed password for root from 65.2.161.68 port 46876 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd[2409]: Failed password for root from 65.2.161.68 port 46898 ssh2
Mar 06:31:41 ip-172-31-35-28 sshd[2399]: Failed password for root from 65.2.161.68 port 46852 ssh2
Mar 06:31:41 ip-172-31-35-28 sshd[2407]: Failed password for root from 65.2.161.68 port 46876 ssh2
Mar 06:31:41 ip-172-31-35-28 sshd[2409]: Failed password for root from 65.2.161.68 port 46898 ssh2
Mar 06:31:42 ip-172-31-35-28 sshd[2423]: Failed password for backup from 65.2.161.68 port 34834 ssh2
Mar 06:31:42 ip-172-31-35-28 sshd[2424]: Failed password for backup from 65.2.161.68 port 34856 ssh2

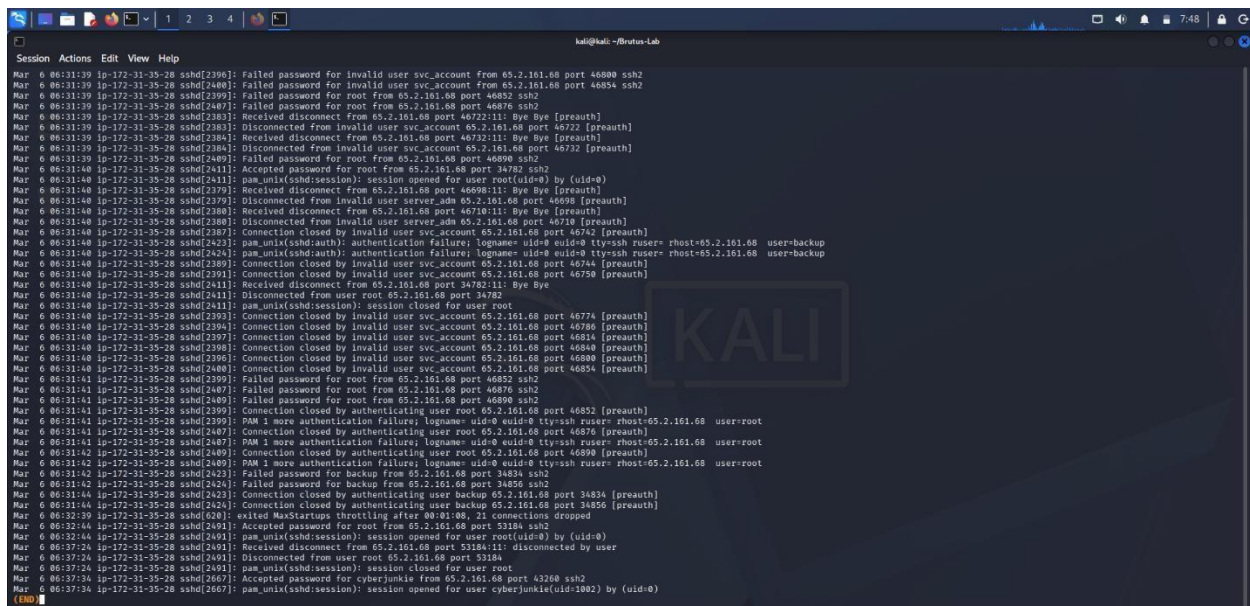
[kali@kali:~/Brutus-Lab]
```

## Step 11 - Reviewing Detailed Authentication Events

After identifying the failed login attempts, I performed a more detailed review of the authentication log to trace the attacker's actions and understand how access was eventually gained.

While reviewing the authentication log file, I observed several important events related to the SSH attack:

- Multiple failed password attempts were made from the IP address **65.2.161.68** against different user accounts such as **root**, **backup**, and **svc\_account**.
- The log contained several **connection closed** and **authentication failure** messages, indicating repeated login attempts by the attacker.
- A successful SSH login was recorded for the **root** account from the same IP address after numerous failed attempts.
- The attacker disconnected from the root session and later established another successful session.
- A successful login was also observed for the **cyberjunkie** account, suggesting that the attacker created or used an additional account to maintain access to the system.



```
Session Actions Edit View Help
Mar 06:31:39 ip-172-31-35-28 sshd(2396): Failed password for invalid user svc_account from 65.2.161.68 port 46800 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd(2400): Failed password for invalid user svc_account from 65.2.161.68 port 46854 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd(2399): Failed password for root from 65.2.161.68 port 46852 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd(2407): Failed password for root from 65.2.161.68 port 46876 ssh2
Mar 06:31:39 ip-172-31-35-28 sshd(2383): Received disconnect from 65.2.161.68 port 46722:11: Bye Bye [preauth]
Mar 06:31:39 ip-172-31-35-28 sshd(2383): Disconnected from invalid user svc_account 65.2.161.68 port 46722 [preauth]
Mar 06:31:39 ip-172-31-35-28 sshd(2384): Received disconnect from 65.2.161.68 port 46732:11: Bye Bye [preauth]
Mar 06:31:39 ip-172-31-35-28 sshd(2384): Disconnected from invalid user svc_account 65.2.161.68 port 46732 [preauth]
Mar 06:31:39 ip-172-31-35-28 sshd(2409): Failed password for root from 65.2.161.68 port 46890 ssh2
Mar 06:31:40 ip-172-31-35-28 sshd(2411): Accepted password for root from 65.2.161.68 port 34782 ssh2
Mar 06:31:40 ip-172-31-35-28 sshd(2411): pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar 06:31:40 ip-172-31-35-28 sshd(2379): Received disconnect from 65.2.161.68 port 46698:11: Bye Bye [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2379): Disconnected from invalid user server_admin 65.2.161.68 port 46698 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2380): Received disconnect from 65.2.161.68 port 46710:11: Bye Bye [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2380): Disconnected from invalid user server_admin 65.2.161.68 port 46710 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2423): pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= root=65.2.161.68 user=backup
Mar 06:31:40 ip-172-31-35-28 sshd(2424): pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= root=65.2.161.68 user=backup
Mar 06:31:40 ip-172-31-35-28 sshd(2391): Connection closed by invalid user svc_account 65.2.161.68 port 46750 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2411): Received disconnect from 65.2.161.68 port 34782:11: Bye Bye
Mar 06:31:40 ip-172-31-35-28 sshd(2411): Disconnected from user root 65.2.161.68 port 34782
Mar 06:31:40 ip-172-31-35-28 sshd(2393): pam_unix(sshd:session): session closed for user root
Mar 06:31:40 ip-172-31-35-28 sshd(2393): Connection closed by invalid user svc_account 65.2.161.68 port 46774 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2394): Connection closed by invalid user svc_account 65.2.161.68 port 46786 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2397): Connection closed by invalid user svc_account 65.2.161.68 port 46814 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2398): Connection closed by invalid user svc_account 65.2.161.68 port 46840 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2396): Connection closed by invalid user svc_account 65.2.161.68 port 46800 [preauth]
Mar 06:31:40 ip-172-31-35-28 sshd(2400): Connection closed by invalid user svc_account 65.2.161.68 port 46854 [preauth]
Mar 06:31:41 ip-172-31-35-28 sshd(2399): Failed password for root from 65.2.161.68 port 46852 ssh2
Mar 06:31:41 ip-172-31-35-28 sshd(2407): Failed password for root from 65.2.161.68 port 46876 ssh2
Mar 06:31:41 ip-172-31-35-28 sshd(2399): Connection closed by authenticating user root 65.2.161.68 port 46832 [preauth]
Mar 06:31:41 ip-172-31-35-28 sshd(2399): PAM 1 more authentication failure; logname=uid=0 euid=0 tty=ssh ruser= root=65.2.161.68 user=root
Mar 06:31:41 ip-172-31-35-28 sshd(2407): Connection closed by authenticating user root 65.2.161.68 port 46876 [preauth]
Mar 06:31:41 ip-172-31-35-28 sshd(2409): PAM 1 more authentication failure; logname=uid=0 euid=0 tty=ssh ruser= root=65.2.161.68 user=root
Mar 06:31:41 ip-172-31-35-28 sshd(2409): PAM 1 more authentication failure; logname=uid=0 euid=0 tty=ssh ruser= root=65.2.161.68 user=root
Mar 06:31:42 ip-172-31-35-28 sshd(2423): Failed password for backup from 65.2.161.68 port 34852 ssh2
Mar 06:31:42 ip-172-31-35-28 sshd(2423): Connection closed by authenticating user backup 65.2.161.68 port 34834 [preauth]
Mar 06:31:42 ip-172-31-35-28 sshd(2424): Connection closed by authenticating user backup 65.2.161.68 port 34850 [preauth]
Mar 06:32:28 ip-172-31-35-28 sshd(500): exited MaxStartups: throttling after 00:01:00, 21 connections dropped
Mar 06:32:44 ip-172-31-35-28 sshd(2491): Accepted password for root from 65.2.161.68 port 53184 ssh2
Mar 06:32:44 ip-172-31-35-28 sshd(2491): pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar 06:32:44 ip-172-31-35-28 sshd(2491): Received disconnect from 65.2.161.68 port 53184:11: disconnected by user
Mar 06:32:44 ip-172-31-35-28 sshd(2491): Disconnected from user root 65.2.161.68 port 53184
Mar 06:37:24 ip-172-31-35-28 sshd(2491): pam_unix(sshd:session): session closed for user root
Mar 06:37:34 ip-172-31-35-28 sshd(2667): Accepted password for cyberjunkie from 65.2.161.68 port 43260 ssh2
Mar 06:37:34 ip-172-31-35-28 sshd(2667): pam_unix(sshd:session): session opened for user cyberjunkie(uid=1002) by (uid=0)
```

## Step 12 - Reviewing Successful Logins and User Sessions

After examining the failed login attempts, I reviewed the authentication and session records to identify successful logins and track user activity on the system.

First, I searched for successful authentication events using the command:

```
grep "Accepted password" auth.log
```

This command displayed all successful SSH login attempts recorded in the log file. From the output, I observed successful logins for the **root** account and the **cyberjunkie** account, including logins originating from the suspicious IP address **65.2.161.68**.

Next, I reviewed SSH-related log entries using the command:

```
grep "sshd" auth.log | less
```

This allowed me to examine SSH authentication activities in greater detail and navigate through the log entries more easily.

Finally, I analyzed the user session records stored in the **wtmp** file using the command:

```
python3 utmp.py wtmp
```

This command displayed historical login and logout sessions. During the review, I identified login sessions associated with the **root** account and the **cyberjunkie** account from the attacker's IP address.

```
Session Actions Edit View Help
Mar 6 06:31:42 ip-172-31-35-28 sshd[2424]: Failed password for backup from 65.2.161.68 port 34856 ssh2

kali@kali:~/Brutus-Lab
$ grep "Accepted password" auth.log
Mar 6 06:19:54 ip-172-31-35-28 sshd[1665]: Accepted password for root from 203.101.190.9 port 42825 ssh2
Mar 6 06:31:44 ip-172-31-35-28 sshd[2411]: Accepted password for root from 65.2.161.68 port 34782 ssh2
Mar 6 06:32:44 ip-172-31-35-28 sshd[2491]: Accepted password for root from 65.2.161.68 port 53184 ssh2
Mar 6 06:37:13 ip-172-31-35-28 sshd[2667]: Accepted password for cyberjunkie from 65.2.161.68 port 43200 ssh2

kali@kali:~/Brutus-Lab
$ grep "sshd" auth.log | less
48
kali@kali:~/Brutus-Lab
$ grep "Failed password" auth.log | wc -l
48

kali@kali:~/Brutus-Lab
$ python3 utmp.py wtmp
"type" "pid" "line" "id" "user" "host" "term" "exit" "session" "sec" "usec" "addr"
"BOOT_TIME" "0" "-" "-" "reboot" "6.2.0-1017-aws" "0" "0" "0" "2024/01/25 06:12:17" "885944" "0.0.0.0"
"INIT" "601" "tty50" "ty50" "-" "-" "0" "601" "2024/01/25 06:12:31" "72401" "0.0.0.0"
"LOGIN" "601" "tty50" "ty50" "-" "-" "0" "601" "2024/01/25 06:12:31" "72401" "0.0.0.0"
"INIT" "618" "tty1" "ty1" "-" "-" "0" "618" "2024/01/25 06:12:31" "80342" "0.0.0.0"
"LOGIN" "618" "tty1" "ty1" "LOGIN" "-" "-" "0" "618" "2024/01/25 06:12:31" "80342" "0.0.0.0"
"RUN_LVL" "33" "-" "-" "runlevel" "6.2.0-1017-aws" "0" "0" "0" "2024/01/25 06:12:33" "792454" "0.0.0.0"
"USER" "1284" "pts/0" "ts/0" "ubuntur" "203.101.190.9" "0" "0" "0" "2024/01/25 06:13:18" "354974" "203.101.190.9"
"DEAD" "1284" "pts/0" "-" "-" "0" "0" "0" "2024/01/25 06:15:12" "956114" "0.0.0.0"
"USER" "1483" "pts/0" "ts/0" "root" "203.101.190.9" "0" "0" "0" "2024/01/25 06:15:48" "806925" "203.101.190.9"
"DEAD" "1484" "pts/0" "-" "-" "0" "0" "0" "2024/01/25 07:34:34" "949753" "0.0.0.0"
"USER" "836798" "pts/0" "ts/0" "root" "203.101.190.9" "0" "0" "0" "2024/02/11 05:33:45" "408334" "203.101.190.9"
"INIT" "838568" "tty50" "ty50" "-" "-" "0" "838568" "2024/02/11 05:39:02" "372641" "0.0.0.0"
"LOGIN" "838568" "tty50" "ty50" "LOGIN" "-" "-" "0" "838568" "2024/02/11 05:39:02" "372641" "0.0.0.0"
"DEAD" "838962" "pts/0" "ts/0" "root" "203.101.190.9" "0" "0" "0" "2024/02/11 05:41:11" "708107" "203.101.190.9"
"DEAD" "838966" "pts/0" "-" "-" "0" "0" "0" "2024/02/11 05:41:45" "712084" "0.0.0.0"
"USER" "842171" "pts/0" "ts/0" "root" "203.101.190.9" "0" "0" "0" "2024/02/11 05:54:27" "775434" "203.101.190.9"
"DEAD" "842073" "pts/0" "-" "-" "0" "0" "0" "2024/02/11 06:08:04" "769516" "0.0.0.0"
"DEAD" "836694" "pts/0" "-" "-" "0" "0" "0" "2024/02/11 06:08:04" "759933" "0.0.0.0"
"RUN_LVL" "0" "-" "-" "shutdown" "6.2.0-1017-aws" "0" "0" "0" "2024/02/11 06:09:18" "731" "0.0.0.0"
"BOOT_TIME" "0" "-" "-" "reboot" "6.2.0-1018-aws" "0" "0" "0" "2024/03/06 01:17:15" "744575" "0.0.0.0"
"INIT" "464" "tty50" "ty50" "-" "-" "0" "464" "2024/03/06 01:17:27" "354378" "0.0.0.0"
"LOGIN" "464" "tty50" "ty50" "LOGIN" "-" "-" "0" "464" "2024/03/06 01:17:27" "354378" "0.0.0.0"
"INIT" "585" "tty1" "ty1" "-" "-" "0" "585" "2024/03/06 01:17:27" "449948" "0.0.0.0"
"LOGIN" "585" "tty1" "ty1" "LOGIN" "-" "-" "0" "585" "2024/03/06 01:17:27" "449948" "0.0.0.0"
"RUN_LVL" "33" "-" "-" "runlevel" "6.2.0-1018-aws" "0" "0" "0" "2024/03/06 01:17:29" "538824" "0.0.0.0"
"USER" "1501" "pts/0" "ts/0" "root" "203.101.190.9" "0" "0" "0" "2024/03/06 01:19:55" "151913" "203.101.190.9"
"USER" "2549" "pts/0" "ts/0" "root" "65.2.161.68" "0" "0" "0" "2024/03/06 01:32:45" "30923" "65.2.161.68"
"DEAD" "2491" "pts/0" "-" "-" "0" "0" "0" "2024/03/06 01:37:24" "598579" "0.0.0.0"
"USER" "2667" "pts/0" "ts/0" "cyberjunkie" "65.2.161.68" "0" "0" "0" "2024/03/06 01:37:35" "475575" "65.2.161.68"
```



## Step 13 - Investigating Sudo Activity and Session Events

After verifying the successful login sessions, I examined the **auth.log** file to identify activities performed with elevated privileges and to review user session events.

First, I searched for sudo-related activities using the command:  
**grep "sudo" auth.log**

The output showed that the **cyberjunkie** account was added to the **sudo** group and later executed commands with root privileges. I also observed the execution of a command that downloaded a script from an external source.

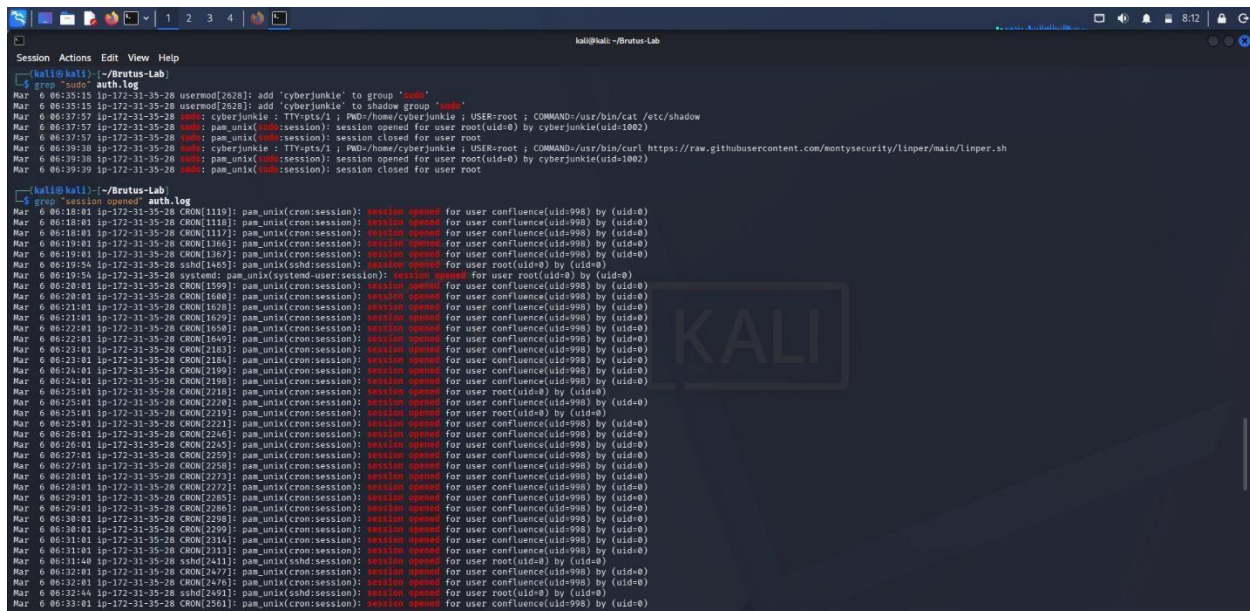
Next, I reviewed session start events using the command:  
**grep "session opened" auth.log**

This command displayed records of user sessions that were successfully opened on the system. The output helped me identify when different users accessed the server and when new sessions were created.

After analyzing the session start records, I reviewed the session termination events to determine when user activities ended on the system using the command:

**grep "session closed" auth.log**

This command displayed all session closure events recorded in the authentication log. The output showed multiple sessions being closed for different users, including both system users and SSH login sessions.



```
kali@kali:~/Brutus-Lab
$ grep "session opened" auth.log
Mar 06:18:01 ip-172-31-35-28 CRON[1119]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:18:01 ip-172-31-35-28 CRON[1110]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:18:01 ip-172-31-35-28 CRON[1117]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:18:01 ip-172-31-35-28 CRON[1106]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:18:01 ip-172-31-35-28 CRON[1167]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:19:54 ip-172-31-35-28 sshd[1405]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar 06:19:54 ip-172-31-35-28 systemd: pam_unix(systemd-user:session): session opened for user root(uid=0) by (uid=0)
Mar 06:20:01 ip-172-31-35-28 CRON[1599]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:20:01 ip-172-31-35-28 CRON[1608]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:21:01 ip-172-31-35-28 CRON[1628]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:21:01 ip-172-31-35-28 CRON[1629]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:22:01 ip-172-31-35-28 CRON[1650]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:22:01 ip-172-31-35-28 CRON[1649]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:23:01 ip-172-31-35-28 CRON[2183]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:23:01 ip-172-31-35-28 CRON[2184]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:24:01 ip-172-31-35-28 CRON[2199]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:24:01 ip-172-31-35-28 CRON[2198]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:25:01 ip-172-31-35-28 CRON[2213]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
Mar 06:25:01 ip-172-31-35-28 CRON[2202]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:25:01 ip-172-31-35-28 CRON[2219]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
Mar 06:26:01 ip-172-31-35-28 CRON[2246]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:26:01 ip-172-31-35-28 CRON[2245]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:27:01 ip-172-31-35-28 CRON[2259]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:27:01 ip-172-31-35-28 CRON[2258]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:28:01 ip-172-31-35-28 CRON[2273]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:28:01 ip-172-31-35-28 CRON[2272]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:29:01 ip-172-31-35-28 CRON[2285]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:29:01 ip-172-31-35-28 CRON[2286]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:30:01 ip-172-31-35-28 CRON[2299]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:31:01 ip-172-31-35-28 CRON[2316]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:31:01 ip-172-31-35-28 CRON[2315]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:31:40 ip-172-31-35-28 sshd[2411]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar 06:32:01 ip-172-31-35-28 CRON[2477]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:32:01 ip-172-31-35-28 CRON[2476]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
Mar 06:32:44 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session opened for user root(uid=0) by (uid=0)
Mar 06:33:01 ip-172-31-35-28 CRON[2561]: pam_unix(cron:session): session opened for user confluence(uid=998) by (uid=0)
```

```
kali@kali: ~/Brutus-Lab
Session Actions Edit View Help
kali@kali:~/Brutus-Lab
$ grep "session closed" auth.log
Mar 06:18:01 ip-172-31-35-28 CRON[1110]: pam_unix(cron:session): session closed for user confuence
Mar 06:18:01 ip-172-31-35-28 CRON[1109]: pam_unix(cron:session): session closed for user confuence
Mar 06:18:01 ip-172-31-35-28 CRON[1107]: pam_unix(cron:session): session closed for user confuence
Mar 06:19:01 ip-172-31-35-28 CRON[1063]: pam_unix(cron:session): session closed for user confuence
Mar 06:19:01 ip-172-31-35-28 CRON[1067]: pam_unix(cron:session): session closed for user confuence
Mar 06:20:01 ip-172-31-35-28 CRON[1099]: pam_unix(cron:session): session closed for user confuence
Mar 06:20:01 ip-172-31-35-28 CRON[1089]: pam_unix(cron:session): session closed for user confuence
Mar 06:21:01 ip-172-31-35-28 CRON[1029]: pam_unix(cron:session): session closed for user confuence
Mar 06:21:01 ip-172-31-35-28 CRON[1028]: pam_unix(cron:session): session closed for user confuence
Mar 06:22:01 ip-172-31-35-28 CRON[1049]: pam_unix(cron:session): session closed for user confuence
Mar 06:22:01 ip-172-31-35-28 CRON[1050]: pam_unix(cron:session): session closed for user confuence
Mar 06:22:01 ip-172-31-35-28 CRON[1083]: pam_unix(cron:session): session closed for user confuence
Mar 06:22:01 ip-172-31-35-28 CRON[1061]: pam_unix(cron:session): session closed for user confuence
Mar 06:24:01 ip-172-31-35-28 CRON[2199]: pam_unix(cron:session): session closed for user confuence
Mar 06:24:01 ip-172-31-35-28 CRON[2198]: pam_unix(cron:session): session closed for user confuence
Mar 06:25:01 ip-172-31-35-28 CRON[2220]: pam_unix(cron:session): session closed for user confuence
Mar 06:25:01 ip-172-31-35-28 CRON[2221]: pam_unix(cron:session): session closed for user confuence
Mar 06:25:01 ip-172-31-35-28 CRON[2219]: pam_unix(cron:session): session closed for user root
Mar 06:25:01 ip-172-31-35-28 CRON[2218]: pam_unix(cron:session): session closed for user root
Mar 06:26:01 ip-172-31-35-28 CRON[2246]: pam_unix(cron:session): session closed for user confuence
Mar 06:26:01 ip-172-31-35-28 CRON[2245]: pam_unix(cron:session): session closed for user confuence
Mar 06:27:01 ip-172-31-35-28 CRON[2258]: pam_unix(cron:session): session closed for user confuence
Mar 06:27:01 ip-172-31-35-28 CRON[2259]: pam_unix(cron:session): session closed for user confuence
Mar 06:28:01 ip-172-31-35-28 CRON[2272]: pam_unix(cron:session): session closed for user confuence
Mar 06:28:01 ip-172-31-35-28 CRON[2287]: pam_unix(cron:session): session closed for user confuence
Mar 06:29:01 ip-172-31-35-28 CRON[2285]: pam_unix(cron:session): session closed for user confuence
Mar 06:29:01 ip-172-31-35-28 CRON[2286]: pam_unix(cron:session): session closed for user confuence
Mar 06:30:01 ip-172-31-35-28 CRON[2298]: pam_unix(cron:session): session closed for user confuence
Mar 06:30:01 ip-172-31-35-28 CRON[2299]: pam_unix(cron:session): session closed for user confuence
Mar 06:31:01 ip-172-31-35-28 CRON[2314]: pam_unix(cron:session): session closed for user confuence
Mar 06:31:01 ip-172-31-35-28 CRON[2313]: pam_unix(cron:session): session closed for user confuence
Mar 06:31:40 ip-172-31-35-28 sshd[2411]: pam_unix(sshd:session): session closed for user root
Mar 06:32:01 ip-172-31-35-28 CRON[2476]: pam_unix(cron:session): session closed for user confuence
Mar 06:32:01 ip-172-31-35-28 CRON[2477]: pam_unix(cron:session): session closed for user confuence
Mar 06:33:01 ip-172-31-35-28 CRON[2501]: pam_unix(cron:session): session closed for user confuence
Mar 06:33:01 ip-172-31-35-28 CRON[2502]: pam_unix(cron:session): session closed for user confuence
Mar 06:34:01 ip-172-31-35-28 CRON[2575]: pam_unix(cron:session): session closed for user confuence
Mar 06:34:01 ip-172-31-35-28 CRON[2576]: pam_unix(cron:session): session closed for user confuence
Mar 06:35:01 ip-172-31-35-28 CRON[2615]: pam_unix(cron:session): session closed for user root
Mar 06:35:01 ip-172-31-35-28 CRON[2616]: pam_unix(cron:session): session closed for user confuence
Mar 06:35:01 ip-172-31-35-28 CRON[2615]: pam_unix(cron:session): session closed for user confuence
Mar 06:36:01 ip-172-31-35-28 CRON[2640]: pam_unix(cron:session): session closed for user confuence
Mar 06:37:01 ip-172-31-35-28 CRON[2632]: pam_unix(cron:session): session closed for user confuence
Mar 06:37:24 ip-172-31-35-28 sshd[2491]: pam_unix(sshd:session): session closed for user root
Mar 06:37:57 ip-172-31-35-28 sudo: pam_unix(sudo:session): session closed for user root
Mar 06:38:01 ip-172-31-35-28 CRON[2700]: pam_unix(cron:session): session closed for user confuence
Mar 06:38:01 ip-172-31-35-28 CRON[2751]: pam_unix(cron:session): session closed for user confuence
```

## Step 14 -Tracking Attacker IP Address Activity

After reviewing user sessions and sudo activities, I focused on the suspicious IP address identified during the investigation to understand all actions performed by the attacker.

I used the following command: `grep "65.2.161.68" auth.log`

This command displayed all log entries associated with the IP address **65.2.161.68**. While reviewing the output, I observed multiple failed login attempts against different user accounts, authentication failure messages, connection events, and SSH-related activities originating from the same IP address.

While reviewing the output related to the IP address **65.2.161.68**, I observed the following:

- Multiple login attempts were made from the same IP address against different user accounts such as **admin** and **backup**.
- Several **authentication failure** messages were recorded, indicating unsuccessful login attempts.
- The logs contained numerous failed password entries, showing evidence of a brute-force attack.
- Multiple connection and disconnection events were observed from the same IP address.
- The attacker repeatedly targeted different usernames in an attempt to gain access to the server.
- All suspicious activities were linked to the IP address **65.2.161.68**, making it the primary source of the attack.

The results confirmed that this IP address was responsible for the brute-force attack and subsequent access to the system. By analyzing all events related to this IP address, I was able to trace the attacker's activities and verify the source of the compromise.

```
Session Actions Edit View Help
kali@kali: ~/Brutus-Lab
$ grep "65.2.161.66" auth.log
Mar 06:11:31 ip-172-31-35-28 sshd[2325]: Invalid user admin from 65.2.161.66 port 46380
Mar 06:11:31 ip-172-31-35-28 sshd[2325]: Received disconnect from 65.2.161.66: port 46380:11: Bye Bye [preauth]
Mar 06:11:31 ip-172-31-35-28 sshd[2325]: Disconnected from invalid user admin 65.2.161.66 port 46380 [preauth]
Mar 06:11:31 ip-172-31-35-28 sshd[628]: drop connection #10 from [65.2.161.66]:46482 on [172.31.35.28]:22 past MaxStartups
Mar 06:11:31 ip-172-31-35-28 sshd[2271]: Invalid user admin from 65.2.161.66 port 46392
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Invalid user admin from 65.2.161.66 port 46444
Mar 06:11:31 ip-172-31-35-28 sshd[2331]: Invalid user admin from 65.2.161.66 port 46436
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2331]: Invalid user admin from 65.2.161.66 port 46422
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Invalid user admin from 65.2.161.66 port 46498
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Invalid user admin from 65.2.161.66 port 46460
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Invalid user admin from 65.2.161.66 port 46454
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Invalid user admin from 65.2.161.66 port 46414
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2333]: Invalid user admin from 65.2.161.66 port 46452
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2333]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh ruser= rhost=65.2.161.66 user=backup
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Failed password for invalid user admin from 65.2.161.66 port 46392 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2331]: Failed password for invalid user admin from 65.2.161.66 port 46436 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Failed password for invalid user admin from 65.2.161.66 port 46444 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2331]: Failed password for invalid user admin from 65.2.161.66 port 46460 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2327]: Failed password for invalid user admin from 65.2.161.66 port 46498 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: Failed password for invalid user admin from 65.2.161.66 port 46454 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: Failed password for backup from 65.2.161.66 port 46452 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: Failed password for backup from 65.2.161.66 port 46468 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2336]: Failed password for invalid user admin from 65.2.161.66 port 46422 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2329]: Failed password for invalid user admin from 65.2.161.66 port 46398 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2329]: Failed password for invalid user admin from 65.2.161.66 port 46414 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2333]: Failed password for invalid user admin from 65.2.161.66 port 46452 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2329]: Failed password for backup from 65.2.161.66 port 46568 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2351]: Failed password for backup from 65.2.161.66 port 46538 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2355]: Failed password for backup from 65.2.161.66 port 46576 ssh2
Mar 06:11:31 ip-172-31-35-28 sshd[2357]: Failed password for backup from 65.2.161.66 port 46582 ssh2
Mar 06:11:35 ip-172-31-35-28 sshd[2327]: Received disconnect from 65.2.161.66 port 46392:11: Bye Bye [preauth]
Mar 06:11:35 ip-172-31-35-28 sshd[2327]: Disconnected from invalid user admin 65.2.161.66 port 46392 [preauth]
Mar 06:11:35 ip-172-31-35-28 sshd[2332]: Received disconnect from 65.2.161.66 port 46444:11: Bye Bye [preauth]
```

## Step 15 - Completing the Investigation and Solving the Challenge

After analyzing the authentication logs, session records, user activities, sudo events, and attacker IP address, I reviewed all collected evidence to answer the Brutus Sherlock challenge questions.



### You have solved Brutus!

Congratulations  **bmanideep1716** best of luck in capturing flags ahead!

|               |                    |                |
|---------------|--------------------|----------------|
| <b>#34018</b> | <b>12 Jun 2026</b> | <b>Retired</b> |
| Sherlock Rank | Solve Date         | Sherlock State |

Ok

Share

After completing the log analysis and answering all challenge questions, I successfully solved the Brutus Sherlock challenge. The platform displayed a completion message confirming that all required objectives had been achieved. This confirmed that all challenge questions were answered correctly and the investigation was completed successfully.

## 5. Skills and Competencies Gained

- Improved understanding of SSH brute-force attacks and how they are detected.
- Learned how to analyze Linux authentication logs such as **auth.log**.
- Developed skills in identifying failed and successful login attempts.
- Enhanced log analysis skills by creating, reviewing, and validating test security logs.
- Improved Linux command-line skills by using log analysis commands such as **grep**, **head** and **tail**.
- Gained experience in tracking attacker activities using IP addresses and log records.
- Developed basic digital forensics and incident investigation skills.

## 6. Feedback and Evidence

### Feedback

- The Brutus Sherlock challenge provided practical experience in investigating a real-world SSH brute-force attack.
- The challenge helped me understand how attackers attempt to gain unauthorized access to Linux systems.
- Analyzing authentication logs improved my log investigation and incident response skills.
- The hands-on approach made it easier to understand cybersecurity concepts compared to theory alone.
- The challenge increased my confidence in using Linux commands for forensic analysis.
- Overall, the investigation was informative and helped improve my cybersecurity knowledge.



## Evidence Collected

- Authentication log (**auth.log**) containing failed and successful SSH login attempts.
- Records showing multiple failed password attempts from the attacker IP address **65.2.161.68**.
- Successful login records for the **root** and **cyberjunkie** accounts.
- User session information obtained from the **wtmp** file.
- Sudo activity records showing actions performed with elevated privileges.
- Session opened and session closed events recorded in the authentication logs.
- Log entries associated with the attacker IP address used during the brute-force attack.
- Challenge completion screenshot confirming successful completion of the Brutus Sherlock challenge.

## 7. Challenges and Solutions

During this project, I faced a few challenges while working on the Brutus Sherlock challenge for the first time. Since I was new to Linux log investigation and SSH attack analysis, understanding the large amount of authentication records in the log files was initially difficult. To overcome this, I learned how to use Linux commands such as **head**, **tail** and **grep** to filter the logs and focus on the most relevant events.

Another challenge was identifying the attacker's activities among numerous login attempts and session records. At first, it was difficult to determine which events were related to the brute-force attack. I solved this by carefully reviewing the authentication logs step by step and tracking failed logins, successful logins, session events and user activities.

I also faced challenges while extracting the evidence files because some files could not be extracted properly using the default extraction method. To resolve this issue, I installed and used the **7-Zip** utility, which allowed me to successfully access all files required for the investigation.

Tracking the attacker's actions after gaining access was another challenge. To understand what happened after the successful login, I analyzed session records, sudo activities and user account information.

Although these challenges required additional time and effort, they helped me improve my Linux log analysis skills, investigation techniques and understanding of SSH brute-force attacks.

## 8. Outcomes and Impact

This project provided me with valuable hands-on experience in cybersecurity investigation and log analysis. Before starting the Brutus Sherlock challenge, I had only a basic understanding of how SSH attacks and authentication logs work. Through this investigation, I gained practical experience in analyzing real-world security events and identifying attacker activities.

The outcome of this project was an improvement in my Linux command-line, log analysis, and problem-solving skills. I learned how to use different Linux commands to filter and analyze large amounts of log data efficiently. I also gained experience in tracking attacker activities and reconstructing an attack timeline using available evidence.

## 9. Conclusion

The Brutus Sherlock challenge was a valuable learning experience that helped me understand how cybersecurity investigations are performed using system logs. Throughout the project, I analyzed authentication records, user sessions and attacker activities to investigate an SSH brute-force attack on a Linux server.

By completing this investigation, I improved my Linux command-line, log analysis and problem-solving skills. I also gained practical experience in identifying suspicious activities, tracking attacker actions, and understanding how security incidents can be investigated using forensic evidence.

Overall, this project enhanced my knowledge of cybersecurity, digital forensics, and incident response. The skills and experience gained during this investigation will support my future learning and help me build a stronger foundation in the field of cybersecurity.